Project report · Multi-tenant SaaS analytics platform

Analivo

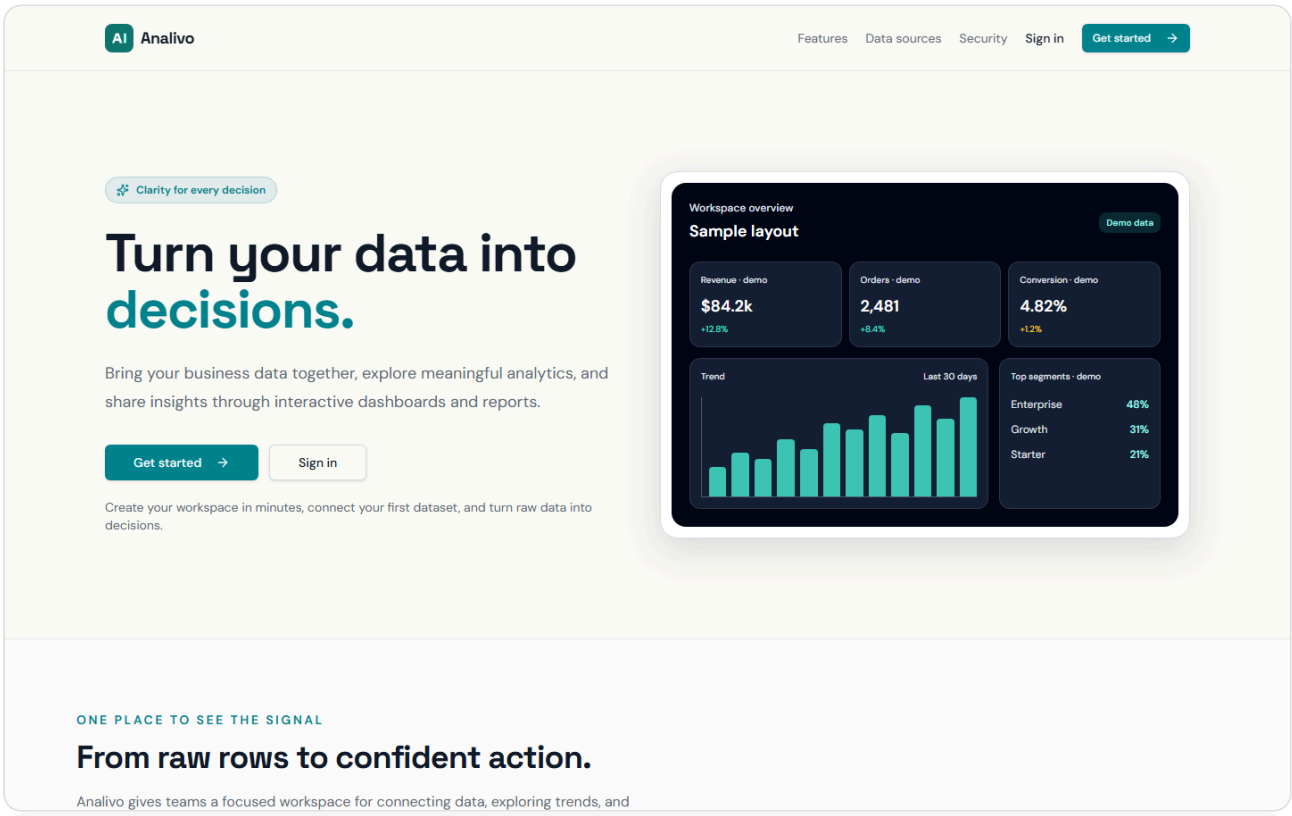
A platform that gives every customer company its own isolated analytics workspace: connect data, ask questions in plain English, and share dashboards, reports, alerts and embeds.

Author
Kishore Balaji P

Date
November 2026

Live
analivo.vercel.app

Code
github.com/KishoreBalajiP/saas-analytics-backend



Contents

1. Executive Summary
2. Objectives and Scope
3. System Architecture
4. Tech Stack Overview
5. Features Overview
6. Folder Structure
7. Key API Endpoints
8. Deployment Overview
9. Implementation & Contributions
10. Challenges & Solutions
11. Future Enhancements
12. Conclusion
13. References

1 Executive Summary

Analivo is a multi-tenant SaaS analytics platform. One backend serves many companies at once, and each company's data, users and permissions stay fully separated.

Inside its workspace, a company connects data sources such as spreadsheets, MongoDB databases and webhooks. It can then explore that data through a tenant-scoped analytics engine, ask questions in plain English, and publish the results as dashboards, scheduled reports, threshold alerts, embeds or a programmatic API. Heavy work such as data syncs and report generation runs on background queues, so the API stays responsive.

145	6	4	6
REST endpoints	built-in RBAC roles	connector types	visualization types

2 Objectives and Scope

Objectives

- **Isolated workspaces.** Each customer company gets its own workspace on a shared platform, and its data can never leak to another tenant.
- **Bring your own data.** Connect spreadsheets, databases and live webhooks without engineering help.
- **Self-serve answers.** Let non-technical users ask questions in plain English and get a chart back.
- **Share insights.** Publish dashboards, schedule reports, raise alerts, and embed analytics in other products.
- **Run as a business.** Support subscription plans, feature entitlements, billing and an admin portal.

Scope

In scope: the tenant and admin portals, data connectors and webhooks, the analytics engine, dashboards and widgets, reports, alerts, notifications, AI-assisted questions, embeds, an external API, billing, and governance and audit.

Out of scope for this version: joins across datasets, drag-and-drop dashboard layout, and performance benchmarking.

3 System Architecture

The backend is one Node.js codebase that runs as two processes: an **HTTP API** and a **worker**. Every request goes through the same chain of middleware before it reaches a feature module. Long-running work is pushed onto Redis queues and handled by the worker.

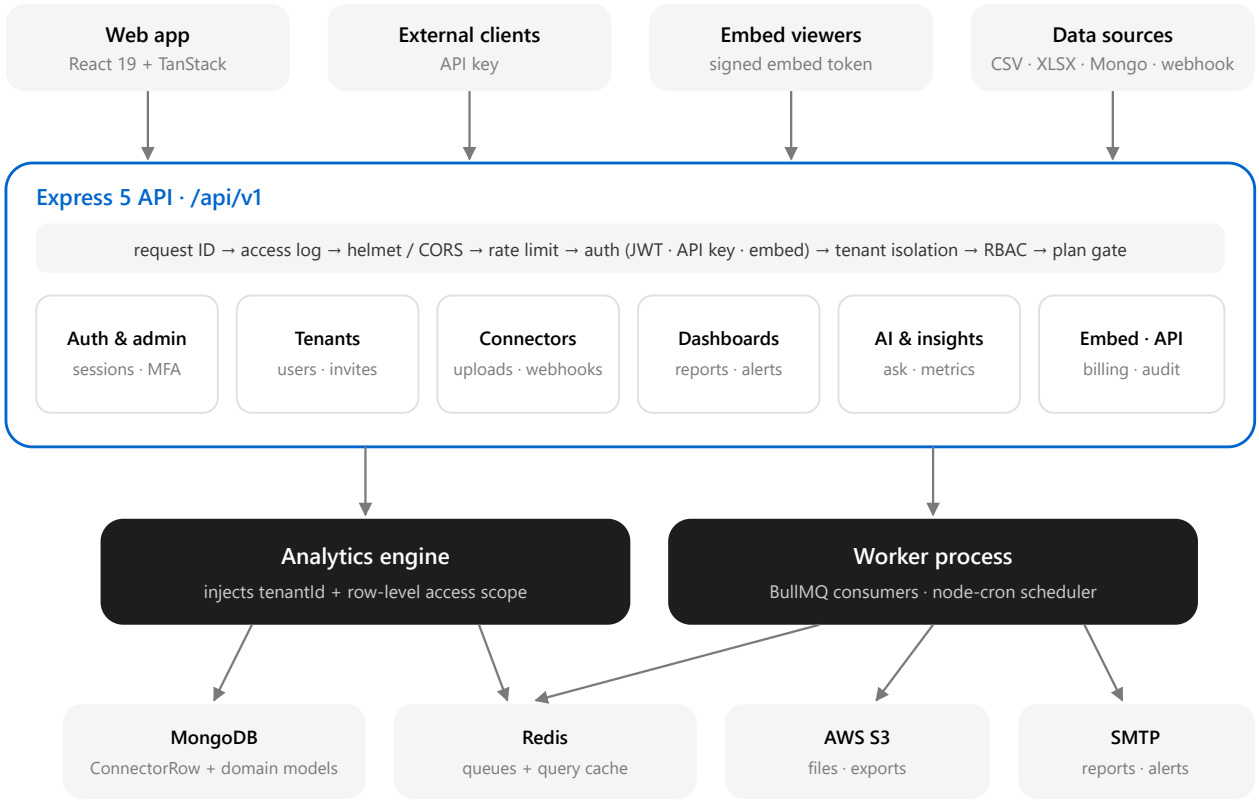


Figure 1. System architecture.

Data flow: from raw rows to a chart

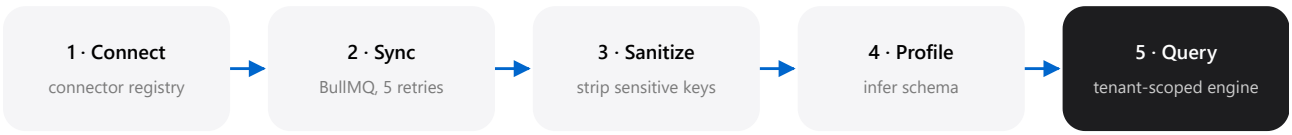


Figure 2. Ingestion pipeline. Each stage has a single owner in the code.

AI pipeline: asking in plain English

The language model never queries the database directly. It proposes a query plan, and that plan has to pass the same validation as a hand-written query before anything runs.

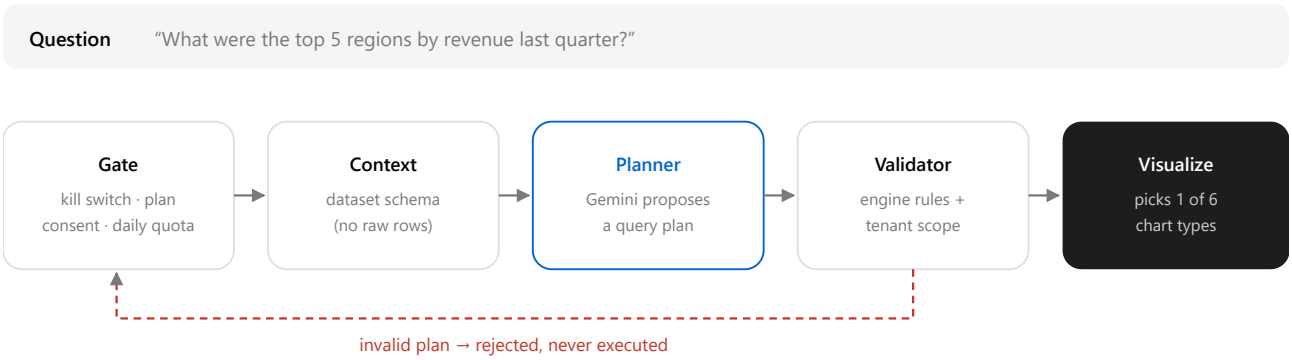


Figure 3. Natural-language query pipeline. Each step is its own module, coordinated by `ask.service.js`.

Security model: isolation is enforced on the server

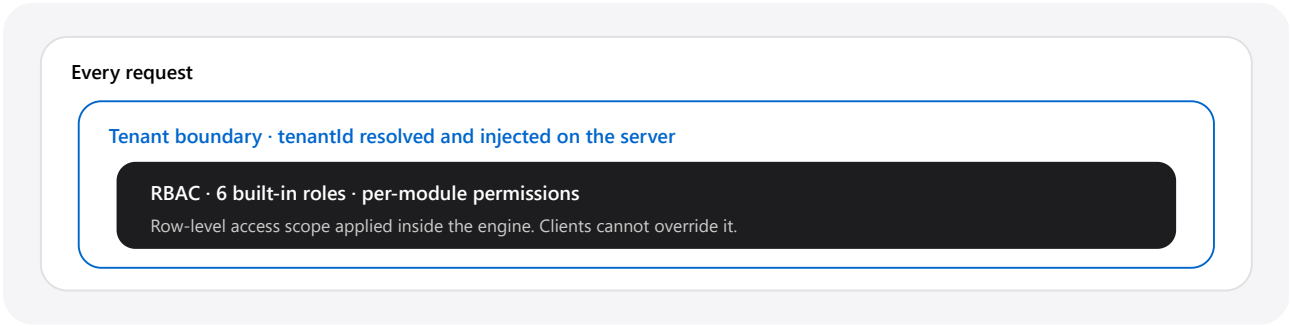


Figure 4. Nested access controls. The front end's route guards only control navigation.

4 Tech Stack Overview

Frontend

- React 19
- TanStack Router and TanStack Query
- Vite and TypeScript
- Tailwind CSS
- Socket.IO client for live updates

Backend

- Node.js 20 (ES modules)
- Express 5
- MongoDB with Mongoose
- Redis, BullMQ and node-cron
- Pino structured logging

External services

- Google Gemini (LLM, configurable provider)
- Razorpay (subscriptions and billing webhooks)
- SMTP for reports, alerts and invitations
- AWS S3 for file storage

Quality & tooling

- Node test runner with mongodb-memory-server
- 5 custom CI guard scripts
- AI and insights evaluation scripts
- Vitest, ESLint and type checks on the front end

5 Features Overview

5.1 Tenant-facing features

- **Workspaces and teams.** Invitations, access requests, and 6 built-in roles with per-module permissions.
- **Data connectors.** CSV and XLSX uploads, MongoDB, and inbound webhooks, with background syncs and automatic schema profiling.
- **Analytics.** Filtering, grouping and aggregation through a validated, tenant-scoped query engine.
- **Ask AI.** Plain-English questions are turned into validated queries and the right chart.
- **Dashboards and widgets.** 6 visualization types, with a publish step that checks a dashboard is ready.
- **Reports and alerts.** Scheduled reports by email and threshold alerts with notifications.
- **Distribution.** Embeds with signed tokens, data exports, and a versioned external API with API keys.
- **Security.** MFA, step-up checks for sensitive actions, and workspace security settings.

5.2 Admin and system features

- A platform admin portal served by the same backend and separated by RBAC
- Subscription plans, entitlements, feature flags and tenant deletion workflows
- Audit logs, access logs, compliance tooling, security alerts and monitoring
- Data retention, backup restore drills and usage reconciliation scripts
- Support sessions, so staff can help a tenant with permission and an audit trail

6 Folder Structure

```
backend/src/
  app.js, server.js   HTTP process
  worker.js           queue + cron process
  routes/             route modules
  middleware/         auth, tenant, RBAC, limits
  controllers/        request handling
  services/           business logic, AI
  repositories/       data access
  models/             Mongoose schemas
  connectors/         connector registry
  queues/  jobs/      BullMQ + scheduled jobs
  cache/   storage/   Redis cache, S3/local
  websocket/          Socket.IO server
  validators/ openapi/
```

```
frontend/src/
  router.tsx          TanStack Router
  routes/             pages
  features/           per-feature api + UI
  components/         shared UI
  hooks/  lib/        API client, utils
  content/            marketing copy

backend/scripts/ci/   guards, migrations,
                      evals, restore drill
backend/tests/        test suite
```

7 Key API Endpoints

All routes are mounted under `/api/v1`. The table shows the main route groups. Together they make up 145 endpoints.

Group	Routes	Purpose
Auth	`/auth`, `/admin-auth`	Sign-up, sign-in, sessions, MFA, admin sign-in
Tenancy	`/tenants`, `/workspace`, `/users`, `/invitations`	Workspaces, members, invitations, settings
Access	`/roles`, `/permissions`, `/access-requests`	RBAC roles and permission management
Data	`/connectors`, `/webhooks`, `/files`	Create sources, upload files, trigger syncs
Analytics	`/analytics`, `/metrics`, `/insights`	Run queries, saved metrics, automatic insights
AI	`/ai`	Natural-language questions
Publishing	`/dashboards`, `/reports`, `/alerts`	Dashboards, widgets, schedules, thresholds
Distribution	`/embed`, `/external/v1`, `/api-keys`, `/data-exports`	Embeds, programmatic API, exports
Billing	`/tenants/:id` subscription, `/billing/webhook`	Plans, Razorpay webhooks
Governance	`/audit-logs`, `/access-logs`, `/compliance`	Audit trail and compliance
Admin	`/admin/*`	Plans, entitlements, subscriptions, tenant deletions, security alerts
Ops	`/health`, `/monitoring`	Health checks and monitoring

8 Deployment Overview

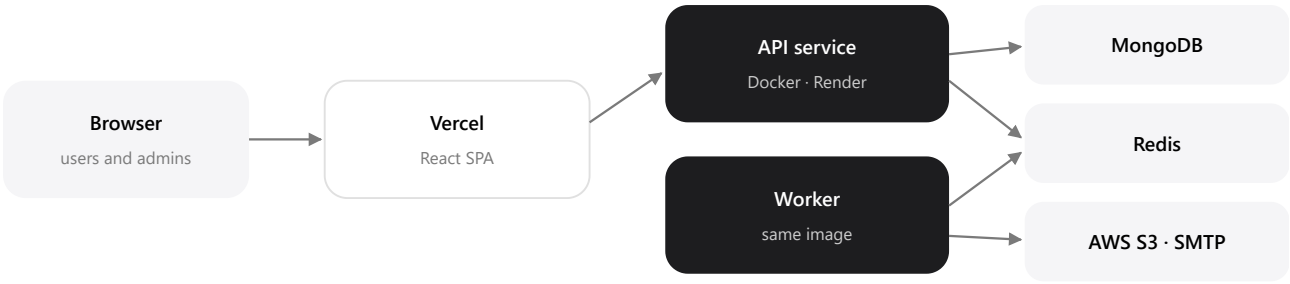


Figure 5. Deployment topology.

- **Frontend:** Vercel
- **Backend:** one Docker image, run as an API service and as a worker
- **Data:** MongoDB, with Redis for queues and caching
- **Files and email:** AWS S3 and SMTP
- **Quality gates:** the test suite and 5 CI guards must pass before a merge

9 Implementation & Contributions

As the sole developer of Analivo, I:

- Designed the multi-tenant architecture, including the tenant isolation and RBAC model
- Built the 145-endpoint REST API with Express 5, input validation and an OpenAPI description
- Wrote the analytics engine, which injects the tenant and row-level access scope into every query
- Built the connector registry, sync queues, sanitisation and schema profiling
- Implemented the guarded AI pipeline: gate, context, planner, validator and visualisation
- Added dashboards, scheduled reports, alerts, embeds, the external API and billing
- Built the React front end with TanStack Router and TanStack Query
- Wrote the test suite, CI guards, data migrations and a backup restore drill

561	5/5	2	1
passing backend tests*	CI guards passing*	processes, one codebase	backend for both portals

* As recorded in the repository on 15 October 2026.

10 Challenges & Solutions

Challenge	Solution
Keeping tenants apart	The engine adds `tenantId` and the access scope as internal keys that a client cannot set, and the check runs on every query, not only in the UI.
An LLM writing queries	The model only proposes a plan, and the plan must pass the same validator as a normal query. It never sees raw rows, and a kill switch, a daily quota and consent checks come first.
Slow, unreliable syncs	Syncs run on BullMQ queues with retries and backoff, and capped upload sizes and rows per sync, so the API stays fast.
Leaking sensitive data	Sensitive fields are stripped when data is ingested and again at query time. Credentials and MFA secrets are encrypted, and API keys are stored only as hashes.
SSRF through connectors	Connectors cannot reach private networks in production.
Docs and code drifting apart	Custom CI guards fail the build when routes, models, config or the README fall out of sync.

11 Future Enhancements

- Joins across datasets in the analytics engine
- A drag-and-drop dashboard layout
- Load testing and published performance benchmarks
- More connectors, such as PostgreSQL, Google Sheets and REST APIs
- Activating the provider integrations that are stubbed today, once they are verified in staging

12 Conclusion

Analivo shows how to build a complete multi-tenant SaaS product from the ground up. Security is built into the request path, slow work runs on queues instead of in requests, and the language model is useful without ever bypassing the rules. The modular design leaves room to add joins, more connectors and scale testing without restructuring the platform.

13 References

1. Express 5 documentation — expressjs.com
2. BullMQ documentation — docs.bullmq.io
3. Mongoose documentation — mongoosejs.com/docs
4. TanStack Router and Query — tanstack.com
5. OWASP Multi-Tenant Security Cheat Sheet — cheatsheetseries.owasp.org
6. Google Gemini API — ai.google.dev
7. Razorpay Subscriptions — razorpay.com/docs